

Assignment 5: RNN/LSTM for Sentiment & Text Generation

(strict plagiarism and ai generated code/content policy in case of detection assignment will be rejected and can lead to serious disciplinary actions including certification and refund)

Title

Building Recurrent Neural Networks for Sentiment Classification and Creative Text Generation

Problem Statement

Design and implement LSTM/GRU-based models for two distinct NLP tasks: (1) sentiment classification on movie reviews, demonstrating the power of sequential modeling, and (2) character-level text generation for creative poetry/lyrics creation. This assignment explores how recurrent architectures capture temporal dependencies and generate coherent sequences.

Dataset

Part A: Sentiment Classification

- **Primary:** [IMDB Movie Reviews](#)
- **Alternative:** [Stanford Sentiment Treebank \(SST-2\)](#)
- **Description:** Binary sentiment (positive/negative)

Part B: Text Generation

- **Primary:** [Poetry Foundation Corpus](#)
- **Alternative:**
 - Song lyrics: [Genius Lyrics Dataset](#)
 - Shakespeare: [Tiny Shakespeare](#)
- **Size:** 1-10 MB text file

Detailed Instructions

PART A: SENTIMENT CLASSIFICATION (50%)

Task A1: Data Preparation (10%)

1. Load IMDB dataset (25k train, 25k test)
2. Preprocessing:
 - Text cleaning (keep sentiment-critical words like "not", "never")
 - Tokenization using Keras Tokenizer

- Build vocabulary (top 10,000 words)
 - Sequence padding (max_length=200 or 500)
3. Create train/validation split (80/20)
 4. Analyze sequence length distribution

Task A2: Baseline Model - Simple RNN (10%)

Python code template: (for help)

```
model = Sequential([  
    Embedding(vocab_size, 128, input_length=max_length),  
    SimpleRNN(64, return_sequences=False),  
    Dropout(0.5),  
    Dense(1, activation='sigmoid')  
])
```

1. Train for 10 epochs
2. Observe vanishing gradient issues
3. Document training challenges

Task A3: LSTM Model (15%)

Python code template (for help)

```
model = Sequential([  
    Embedding(vocab_size, 128),  
    LSTM(128, return_sequences=True),  
    Dropout(0.5),  
    LSTM(64),  
    Dropout(0.5),  
    Dense(1, activation='sigmoid')  
])
```

1. Train with early stopping
2. Compare with Simple RNN
3. Visualize training curves

4. *Experiment with:*

- *Number of LSTM layers (1, 2, 3)*
- *Hidden units (64, 128, 256)*
- *Dropout rates (0.3, 0.5, 0.7)*

Task A4: Bidirectional LSTM (15%)

Python code implementation (for help purpose)

```
model = Sequential([  
    Embedding(vocab_size, 128),  
    Bidirectional(LSTM(64, return_sequences=True)),  
    Dropout(0.5),  
    Bidirectional(LSTM(32)),  
    Dense(1, activation='sigmoid')  
])
```

1. *Train and evaluate*
2. *Compare with unidirectional LSTM*
3. *Analyze computational cost vs performance gain*

Task A5: GRU Comparison (10%)

Python code template(for help purpose)

```
model = Sequential([  
    Embedding(vocab_size, 128),  
    GRU(128, return_sequences=True),  
    Dropout(0.5),  
    GRU(64),  
    Dense(1, activation='sigmoid')  
])
```

1. *Train GRU model*
2. *Compare with LSTM:*
 - *Accuracy*
 - *Training time*
 - *Number of parameters*
 - *Memory usage*

PART B: TEXT GENERATION (50%)

Task B1: Character-Level Data Preparation (10%)

1. Load poetry/lyrics corpus
2. Character-level analysis:
 - Unique characters
 - Character frequency distribution
 - Build char-to-index and index-to-char mappings
3. Create training sequences:

Python code template(for help purpose) :

```
seq_length = 100
```

```
sequences = []
```

```
next_chars = []
```

```
for i in range(len(text) - seq_length):
```

```
    sequences.append(text[i:i+seq_length])
```

```
    next_chars.append(text[i+seq_length])
```

4. One-hot encode or use integer encoding

Task B2: Character-Level LSTM (20%)

python

```
model = Sequential([
```

```
    LSTM(256, input_shape=(seq_length, vocab_size), return_sequences=True),
```

```
    Dropout(0.3),
```

```
    LSTM(256),
```

```
    Dropout(0.3),
```

```
    Dense(vocab_size, activation='softmax')
```

```
])
```

1. Train for 30-50 epochs
2. Save checkpoints every 5 epochs
3. Monitor loss convergence
4. Generate sample text during training

Task B3: Text Generation with Sampling (10%)

Implement multiple sampling strategies:

1. **Greedy Sampling** (deterministic):
2. **Temperature-based Sampling:**
3. **Top-k Sampling:**
 - Select from top 5 or 10 most likely characters
4. **Top-p (Nucleus) Sampling:**
 - Select from smallest set of chars with cumulative probability $> p$

Task B4: Generate Creative Text (10%)

1. Generate 500-character samples with different:
 - Seed texts (5 different starting phrases)
 - Temperatures (low=0.5, medium=1.0, high=1.5)
 - Sampling strategies
2. Create comparison grid showing diversity vs coherence
3. Generate 5 complete poems/song verses
4. Qualitative evaluation:
 - Coherence
 - Creativity
 - Grammar
 - Style consistency

PART C: COMPARATIVE ANALYSIS (Across Both Parts)

Task C1: Compare with Assignment 3 Baseline

Create comparison table:

Model	Architecture	Accuracy	Training Time	Parameters	Inference Time
Logistic Regression (TF-IDF)	-	X%	Xs	-	Xms
Simple RNN	1 layer	X%	Xm	Xk	Xms
LSTM	2 layers	X%	Xm	Xk	Xms
Bidirectional LSTM	2 layers	X%	Xm	Xk	Xms
GRU	2 layers	X%	Xm	Xk	Xms

Task C2: Architecture Analysis

1. Visualize model architectures (use plot_model)
2. Compare parameter counts
3. Analyze computational complexity
4. Discuss practical deployment considerations

Expected Outputs

1. Code Deliverables

- sentiment_rnn.py - Sentiment classification models
- text_generation.py - Character-level LSTM for generation
- sampling_strategies.py - Different sampling implementations
- Jupyter notebook with experiments
- Training scripts with configurable hyperparameters

2. Model Files

- Best sentiment classifier (.h5)
- Text generation model checkpoints (epoch 10, 20, 30, final)
- Tokenizers/vocabularies (pickle)

3. **Generated Text Samples**

- 50+ generated text samples
- Organized by temperature and sampling strategy
- Annotated with quality ratings
- Best 10 samples highlighted

4. **Visualizations** (minimum 12)

- Sequence length distribution (IMDB)
- Training/validation loss curves (all models)
- Training/validation accuracy curves
- LSTM vs GRU comparison (bar charts)
- Character frequency distribution
- Generated text quality vs temperature (plot)
- Confusion matrix (sentiment task)
- ROC curve (sentiment task)
- Model architecture diagrams
- Parameter count comparison
- Training time comparison
- Attention weights visualization (bonus)

5. **Technical Report** (5-6 pages)

- **Part A: Sentiment Classification**
 - Architecture evolution (RNN → LSTM → Bi-LSTM → GRU)
 - Training challenges and solutions
 - Comparison with classical ML baseline
 - Hyperparameter tuning insights
- **Part B: Text Generation**
 - Character-level modeling approach
 - Training dynamics and convergence
 - Sampling strategies comparison
 - Qualitative evaluation of generated text

- **Comparative Analysis**
 - RNN variants comparison
 - When to use LSTM vs GRU
 - Practical considerations
- **Conclusion:** Lessons learned, limitations, future improvements

Findings/Observations to Document

1. How does Simple RNN compare to LSTM in handling long sequences?
2. What's the benefit of bidirectional processing for sentiment analysis?
3. How does GRU's simpler architecture compare to LSTM in practice?
4. At what epoch does the text generation model start producing coherent output?
5. How does temperature affect creativity vs coherence trade-off?
6. Which sampling strategy produces the best results?
7. How do RNNs compare to transformers (conceptually)?
8. What types of patterns does the character-level model learn first?
9. Can the model learn rhyme schemes or meter in poetry?
10. What are the main failure modes for each task?

Evaluation Rubric (100 points)

Criteria	Points	Description
Sentiment Models	30	All RNN variants correctly implemented, proper training
Text Generation	25	Functional character-level LSTM, quality text output
Sampling Strategies	15	Multiple sampling methods implemented and compared
Comparative Analysis	15	Thorough comparison of architectures and with baseline
Visualizations	10	Clear, informative plots supporting analysis
Report Quality	5	Well-structured, insightful observations

Bonus Points (10)

- Implement attention mechanism for sentiment task (+4)
- Word-level text generation comparison (+3)
- Generate poetry with specific rhyme scheme (+2)

Target Metrics

- **Sentiment Classification:**
 - Minimum: 85% accuracy
 - Good: 88-90% accuracy
 - Excellent: >90% accuracy
- **Text Generation:**
 - Qualitative: Coherent sentences by epoch 20
 - Loss: <1.5 (character-level cross-entropy)
